The collaboration of these three tiers is mainly used in web applications.

n-tier architecture: Other than the client, application server, and database server, the application server is further divided into multiple tiers, where each server acts as an entry point to do a different kind of processing on the provided data.

Peer-to-peer architecture: The peers are both servers and clients on a network. They work together to provide computing, storage, and network resources to network users. The responsibility for a task is uniformly distributed among the users of the network — these users are called peers.

Applications of distributed systems

Fault-tolerance: In case one of the nodes in the computer network goes down, to handle the current node, the load can be balanced to other subsequent nodes.

Multi-layered: In a distributed architecture, several computers come together over a network to do a specific task; for example, one computer node collects the data at location A, while the other processes it at location B.

Cost-effectiveness: Rather than investing in a single high-end computing device, the same computation can be achieved via clusters of several low-end devices.

Standard problems

In distributed systems, when we say every node is a computing unit, it must have an independent memory when it exchanges information by passing messages to other nodes. The first challenge that arises with this communication is how to do it efficiently so that the message is passed regardless of the network. Since computational problems are typically related to graphs, distributed algorithms are designed to solve them.

Distributed algorithms run on computation units, where each unit is connected to another over a network. These algorithms are well known to solve some of the standard problems in distributed systems, which include leader election, consensus, distributed search, spanning tree generation, mutual exclusion, and resource allocation.

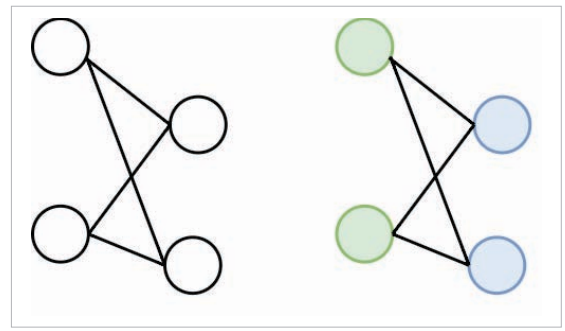

Figure 6: Colouring of a node graph connected in a network

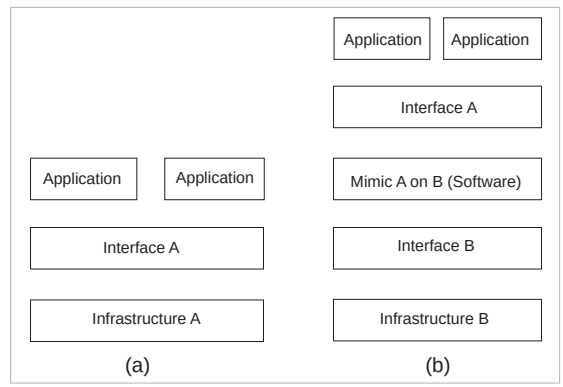

Figure 7: (a) Basic setup between infrastructure, interface, and application (b) Basic setup with virtualization of system A on system B

Let us take an example of a basic computation problem of distributed systems, where we see how a distributed system finds the colouring of a given graph G. As discussed earlier, we visualise the distributed systems model as a graph G, where G is a connection between computing units over a network. Every vertex or node in graph G denotes the computing unit. In finding the colouring of this graph, we see each node of G in a communication link with another node denoted via an edge. Initially, each node only knows about its immediate neighbours in the graph,

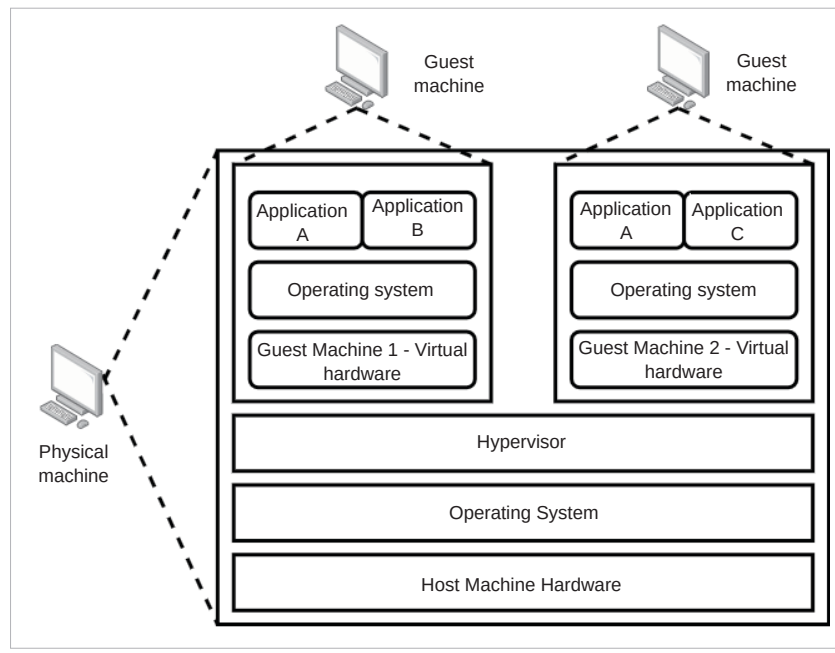

Figure 8: Hardware virtualization